МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования «Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

# Отчет по выполнению практических работ по дисциплине "Функциональное программирование"

Отчет выполнил
студент группы 5130201/40001 _____ Костина А.В.
Отчет принял
преподаватель _____ Моторин Д.Е.

«____»_____ 2026г.

Санкт-Петербург, 2026

# Содержание

**Введение** **3**

**1 Структура репозитория** **4**

**2 График выполнения и сдачи работ** **5**

**3 Описание коммитов** **6**
3.1 Практическая работа №1 . . . . . 6
3.2 Практическая работа №2 . . . . . 6
3.3 Практическая работа №3 . . . . . 7
3.4 Практическая работа №4 . . . . . 7
3.5 Практическая работа №5 . . . . . 7

**4 Защита практических работ** **9**
4.1 Практическая работа №1 . . . . . 9
4.2 Практическая работа №2 . . . . . 9
4.3 Практическая работа №3 . . . . . 9
4.4 Практическая работа №4 . . . . . 10
4.5 Практическая работа №5 . . . . . 10

**Приложение** **11**

# Введение

В отчете представлены результаты выполнения пяти практических работ по курсу «Функциональное программирование». Все работы размещены в закрытом репозитории GitHub: https://github.com/wndlbwzhxxx/KostinaAVFP. Основную часть отчета составляет описание каждого коммита, сделанного в течение семестра.

# 1 Структура репозитория

- **KostinaAVFP**
  - **practice_01** — папка с решением 1 работы
    - Pr01_1.hs — файл решения 1 части
    - pr01_2.hs — файл решения 2 части
  - **practice_02** — папка с решением 2 работы
    - MyEvolProject — cabal-проект (решение 1 части)
    - myParser — stack-проект (решение 2 и 3 части)
    - pr02_1.hs
    - pr02_2.hs
    - pr02_3.hs
  - **practice_03** — папка с решением 3 работы
    - glitcher — stack-проект (решение 1 части)
    - labirint — stack-проект (решение 2 части)
    - pr03_1.hs
    - pr03_2.hs
  - **practice_04** — папка с решением 4 работы
    - labirint-game — stack-проект (решение задания)
    - pr04_1.hs
  - **practice_05** — папка с решением 5 работы
    - quick — stack-проект (решение задания)
    - pr05_1.hs
  - otchet_lab1-5.pdf

# 2 График выполнения и сдачи работ

| Хэш | Дата | Сообщение коммита | ПР |
|---|---|---|---|
| cb83693 | 28.02.2026 | Добавлен каталог первого практического задания | 1 |
| 5b736d4 | 06.03.2026 | Добавлены функции из файла pr01_2.hs | 1 |
| 245c68b | 06.03.2026 | Защита работы | 1 |
| a133d66 | 12.03.2026 | Добавлено выполнение заданий pr02_1.hs | 2 |
| d314155 | 19.03.2026 | Добавлено выполнение заданий pr02_2.hs | 2 |
| b2322bb | 26.03.2026 | Добавлено выполнение заданий pr02_3.hs | 2 |
| 5cc50ad | 03.04.2026 | Переустановлен stack и дополнены многострочные комментарии в MyTree, MyMaybe и MyEither | 2 |
| 4474650 | 05.04.2026 | Защита работы (исправлена версия для cabal) | 2 |
| b88bc21 | 16.04.2026 | Добавлено выполнение заданий pr03_1.hs | 3 |
| b159023 | 16.04.2026 | Добавлено выполнение заданий pr03_2.hs | 3 |
| 087fbea | 23.04.2026 | Создание проекта и реализация MyStateT | 4 |
| b7e0d19 | 23.04.2026 | Добавление структур данных и загрузка лабиринта из файла | 4 |
| 39ef964 | 23.04.2026 | Реализация игровой логики | 4 |
| 8692e0e | 14.05.2026 | настроен проект, добавлен QuickCheck и реализованы функции | 5 |
| 346762a | 14.05.2026 | добавлены тесты QuickCheck для addMod | 5 |
| 840a003 | 14.05.2026 | добавлены генераторы и тесты QuickCheck для reverseWords | 5 |

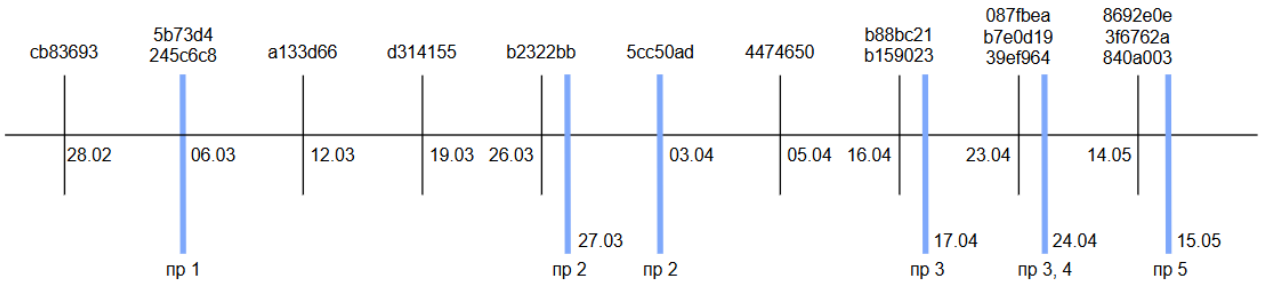



Рис. 1: Временнная диаграмма выполнения и сдачи работ

# 3 Описание коммитов

## 3.1 Практическая работа №1

**cb83693** (20.02.2026) — Добавлен каталог первого практического задания. Реализованы функции для работы с кортежами (`myFST`, `mySND`, `myTHRD`) и списками (`myHead`, `myTail`, `myTake`, `myDrop`, `myProduct`, `myZip`, `myZip3`, `myUnzip`). Реализованы функции высшего порядка (`myFilter`, `myMap`, `myZipWith`, `myZipWith3`, `myAll`, `myAny`, `myComposition`). Код размещён в файле `Pr01_1.hs`.

**5b736d4** (20.02.2026) — Добавлены функции из файла `pr01_2.hs`. Реализованы `myZipSave` (сохранение хвоста более длинного списка), `myUnzipSave` (восстановление списков разной длины), `myReverse` (через свёртки), `myFoldl1`/`myFoldr1` (свёртки для непустых списков), `myTakeWhile` и `mySpan` (через правую свёртку), `myMaybe` (обработка отсутствующего значения), `myMap` для пользовательского типа `MyList`, `myUnFoldr` (развёртка). Расширена предметная область «Торты»: добавлены три новых типа тортов, контроль количества ингредиентов через `newtype` (`Grams`, `Pieces`), обработка недостатка ингредиентов через `Either String (Cake, String)`.

## 3.2 Практическая работа №2

**a133d66** (03.03.2026) — Выполнение части 2.1. Создан `cabal`-проект `MyEvolProject`, добавлен модуль `MyEvolModule.hs`. Определён тип-сумма `MyEvolution` с конструкторами от `LUCA` до `Humans`. Вручную реализованы представители классов типов `Show`, `Read`, `Eq`, `Ord`, `Enum`, `Bounded`. Аналогичный тип `MyEvolution'` с автоматическим выводом экземпляров через `deriving`. В `main` добавлен вывод отсортированного списка значений.

**d314155** (10.03.2026) — Выполнение части 2.2. Создан `stack`-проект `myParser`. В директории `src/MyTypes` созданы модули: `MyTree` (бинарное дерево с реализацией `Foldable`, `Functor`, `Applicative`); `MyMaybe` (`Foldable`, `Semigroup`, `Monoid`, `Functor`, `Applicative`); `MyEither` (`Foldable`, `Semigroup`, `Functor`, `Applicative`). В конец каждого файла добавлены многострочные комментарии с примерами вызова функций из `ghci`.

**b2322bb** (17.03.2026) — Выполнение части 2.3. Создана директория `src/MyParsers`. Реализованы три варианта парсера: ручной (на основе `MyMaybe`), с использованием библиотеки `Parsec` и с использованием `Attoparsec`. Все парсеры разбирают выражения вида `"число*число"` и `"число+число"`. В `Main.hs` организован единый интерфейс вызова всех трёх парсеров.

**5cc50ad** (20.03.2026) — Переустановка `stack` для стабильной работы проекта. Дополнены многострочные комментарии в модулях `MyTree`, `MyMaybe`, `MyEither`.

## 3.3 Практическая работа №3

**b159023** (05.04.2026) — Выполнение части 3.1. Создан проект `glitcher` по генерации глитчей на изображениях (Урок 25.2 из книги «Программируй на Haskell» Уилла Курта). Чистые функции (`replaceByte`, `sortSection`, `reverseSection`) вынесены в `src/Glitch.hs`. Реализованы функции с генерацией случайных чисел: `randomReplaceByte`, `randomSortSection`, `randomReverseBytes`. Для комбинирования эффектов использован `foldM`. Создано 5 версий глитч-эффектов с сохранением в отдельные файлы.

**8c1e60c** (12.04.2026) — Выполнение части 3.2. Реализован поиск маршрута в лабиринте с использованием монады `RWS` (`Reader` для карты лабиринта, `Writer` для лога перемещений, `State` для посещённых комнат). Лабиринт из 10 комнат (локации из Genshin Impact) загружается из карты. Поиск реализован через взаимно рекурсивные функции `findPathFromCurrent` и `tryPaths` (по аналогии с `some`/`many`). `do`-нотация не используется.

## 3.4 Практическая работа №4

**087fbea** (20.04.2026) — Создание `stack`-проекта `labirint-game`. Реализован собственный трансформер монад `MyStateT` с экземплярами `Functor`, `Applicative`, `Monad`, `MonadTrans`, `MonadIO`. Определены функции `get`, `put`, `modify`.

**b7e0d19** (25.04.2026) — Добавлены структуры данных и загрузка лабиринта из файла. Определены типы `Room` (название, описание, направления) и `Maze` (карта). Реализован парсер текстового файла лабиринта. Создан тестовый лабиринт `maze.txt` (10 комнат из Genshin Impact).

**39ef964** (02.05.2026) — Реализована игровая логика с использованием трёх трансформеров: `MyStateT` (состояние: текущая комната, посещённые комнаты, число ходов), `WriterT` (лог перемещений), `ReaderT` (карта лабиринта). Реализованы явное терминирование трансформеров, подъём эффектов через `lift`, игровой цикл с командами `north`/`south`/`west`/`east`/`quit`. При достижении комнаты `finish` игра завершается победой. После игры выводятся количество ходов и полный лог перемещений.

## 3.5 Практическая работа №5

**8692e0e** (08.05.2026) — Создан `stack`-проект `quick`, добавлена зависимость `QuickCheck`. В модуле `Lib.hs` реализованы функции: `addMod :: Int -> Int -> Int -> Int` (сложение по модулю) и `reverseWords :: String -> String` (переворот порядка слов).

**316762a** (08.05.2026) — Добавлены тесты `QuickCheck` для функции `addMod`. Проверяются свойства: корректность сложения по модулю, нейтральный элемент (ноль), коммутативность. Для защиты от деления на ноль используется тип `Positive Int`.

**840a003** (08.05.2026) — Добавлены генераторы осмысленных строк (`genWord` и `genSentence`) и тесты `QuickCheck` для `reverseWords`. Проверяются свойства: переворот пустой строки, переворот строки с одним словом, изменение порядка слов в строке с несколькими словами, двойное применение возвращает исходную строку. Все тесты вынесены в `test/Spec.hs` и проходят успешно.

# 4 Защита практических работ

## 4.1 Практическая работа №1

**Дата защиты:** 06.03.2026

**Вопрос:** Написать функцию с типом `func :: Maybe [a -> b] -> Maybe [a] -> Maybe [b]`

**Решение:**

```
ffun :: Maybe [a -> b] -> Maybe [a] -> Maybe [b]
ffun xs Nothing = Nothing
ffun Nothing ys = Nothing
ffun (Just xs) (Just ys) =  Just (doing xs ys)
where doing [] _ = []
doing _ [] = []
doing (f:fs) (k:ks) = f k : doing fs ks
```

## 4.2 Практическая работа №2

**Дата защиты:** 27.03.2026

**Вопрос:** Что такое полугруппа?

**Ответ:** Тип с ассоциативной бинарной операцией (<>). Единственный закон: (x <> y) <> z = x <> (y <> z).

**Вопрос:** Вписать переменные в выражение :t ( <*> ), чтобы вывод был Either (a -> a) b

**Решение:**

```
:t (Left id <*> Right 3)
```

**Дата защиты:** 03.04.2026

**Вопрос:** Запустить stack проект

**Решение:** Успешно запущен stack проект

## 4.3 Практическая работа №3

**Дата защиты:** 17.04.2026

**Вопрос:** Написать монадический байнд ( _ »= _ ) Reader

**Решение:**

```
r = reader (\x -> x * 2) >>= (\y -> reader (\x -> x + y))
```

**Дата защиты:** 24.04.2026

**Вопрос:** Перечислить монады и их побочные эффекты

**Ответ:**

MonadId - отсутствие эффектов MonadMaybe - завершение неудачей MonadExcep - завершение с ошибкой MonadList - множественные результаты MonadWriter - запись в лог MonadReader - чтение из внешнего окружени MonadState - работа с мутабельным состоянием MonadIO - ввод/вывод (файлы, консоль)

## 4.4 Практическая работа №4

**Дата защиты:** 24.04.2026

**Вопрос:** Почему runGame имеет тип Game () -> Maze -> IO (), если Game параметризован типом a?

**Ответ:** Тип Game a параметризован, чтобы позволить различным вычислениям возвращать разные типы — например, gameTurn :: Game Bool возвращает Bool, getNeighbors :: Game [RoomName] возвращает список комнат. В случае runGame передаётся Game (), потому что gameLoop :: Game () не возвращает содержательного значения — он существует только ради эффектов: изменения состояния (StateT), накопления лога (WriterT) и ввода-вывода (IO). () — стандартный способ выразить «не возвращаемого значения» в Haskell, аналогично void в других языках. При раскрутке стека runMyStateT, runWriterT, runReaderT все обёртки снимаются и в итоге остаётся IO ().

## 4.5 Практическая работа №5

**Дата защиты:** 15.05.2026

**Вопрос:** Как вывести что подается на вход в отдельный тест?

**Ответ:**

```
quickCheck $ verbose prop_addMod_correct
```

**Вопрос:** Как изменить количество тестов?

**Ответ:**

```
quickCheck $ withMaxSuccess 5 prop_addMod_correct
```

# Приложение

## Practice Work No. 1, Part 1 (pr01_1.hs)

```
1     module Pr01_1 where
2
3     myFST :: (a, b, c) -> a
4     myFST (x, y, z) = x
5
6     mySND :: (a, b, c) -> b
7     mySND (x, y, z) = y
8
9     myTHRD :: (a, b, c) -> c
10    myTHRD (x, y, z) = z
11
12    myHead :: [a] -> a
13    myHead [] = error "list is empty"
14    myHead (x : xs) = x
15
16    myTail :: [a] -> [a]
17    myTail [] = error "list is empty"
18    myTail (x : xs) = xs
19
20    myTake :: Int -> [a] -> [a]
21    myTake 0 xs = []
22    myTake n [] = []
23    myTake n (x : xs) = x : myTake (n - 1) xs
24
25    myDrop :: Int -> [a] -> [a]
26    myDrop 0 xs = xs
27    myDrop n [] = []
28    myDrop n (x : xs) = myDrop (n - 1) xs
29
30    myProduct :: [Int] -> Int
31    myProduct [] = error "list is empty"
32    myProduct [xs] = xs
33    myProduct (x : xs) = x * myProduct xs
34
35    myZip :: [a] -> [b] -> [(a, b)]
36    myZip [] ys = []
37    myZip xs [] = []
38    myZip (x : xs) (y : ys) = (x, y) : myZip xs ys
39
40    myZip3 :: [a] -> [b] -> [c] -> [(a, b, c)]
41    myZip3 [] ys zs = []
42    myZip3 xs [] zs = []
43    myZip3 xs ys [] = []
44    myZip3 (x : xs) (y : ys) (z : zs) = (x, y, z) : myZip3 xs ys
      zs
45
46    myUnzip0 :: [(a,b)] -> ([a],[b])
47    myUnzip0 [] = ([], [])
48    myUnzip0 ((x,y): xys) = (x : xs, y : ys)
```

```
49    where (xs, ys) = myUnzip0 xys
50
51    myFilter0 :: (a -> Bool) -> [a] -> [a]
52    myFilter0 predicate [] = []
53    myFilter0 predicate (x : xs)
54     predicate x = x : myFilter0 predicate
    xs otherwise   = myFilter0 predicate xs
55
56    myFilter1 :: (a -> Bool) -> [a] -> [a]
57    myFilter1 predicate [] = []
58    myFilter1 predicate (x : xs) =
59    if predicate x
60    then x : myFilter1 predicate xs
61    else myFilter1 predicate xs
62
63    myMap :: (a -> b) -> [a] -> [b]
64    myMap \_ [] = []
65    myMap f (x : xs) = f x : myMap f xs
66
67    myZipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
68    myZipWith \_ \_ [] = []
69    myZipWith \_ [] \_ = []
70    myZipWith f (x : xs) (y : ys) = f x y : myZipWith f xs ys
71
72    myZipWith3 :: (a -> b -> c -> d) -> [a] -> [b] -> [c] -> [d]
73    myZipWith3 \_ [] \_ \_ = []
74    myZipWith3 \_ \_ [] \_ = []
75    myZipWith3 \_ \_ \_ [] = []
76    myZipWith3 f (x : xs) (y : ys) (z : zs) = f x y z :
    myZipWith3 f xs ys zs
77
78    myAll :: (a -> Bool) -> [a] -> Bool
79    myAll predicate [] = True
80    myAll predicate (x : xs) = predicate x \&\& myAll predicate
    xs
81
82    myAny :: (a -> Bool) -> [a] -> Bool
83    myAny predicate [] = False
84    myAny predicate (x : xs) = predicate x myAny predicate xs
85
86    myComposition :: (b -> c) -> (a -> b) -> a -> c
87    myComposition f g x = f (g x)
88
```

Листинг 1: pr01_1.hs

# Practice Work No. 1, Part 2 (pr01_2.hs)

```
1     module Pr01_2 where
2
3     myZipSave :: [a] -> [b] -> ([(a, b)], Either [a] [b])
4     myZipSave [] [] = ([], Left [])
```

```
5     myZipSave xs [] = ([], Left xs)
6     myZipSave [] ys = ([], Right ys)
7     myZipSave (x:xs) (y:ys) =
8     let (pairs, tail) = myZipSave xs ys
9     in ((x,y) : pairs, tail)
10
11    myUnzipSave :: ([(a, b)], Either [a] [b]) -> ([a],[b])
12    myUnzipSave ([], Left []) = ([], [])
13    myUnzipSave ([], Left xs) = (xs, [])
14    myUnzipSave ([], Right ys) = ([], ys)
15    myUnzipSave ((x,y):xys, tail) =
16    let (xs, ys) = myUnzipSave (xys, tail)
17    in (x:xs, y:ys)
18
19    myReverse :: [a] -> [a]
20    myReverse xs = foldl (flip(:)) [] xs
21
22    myFoldl1 :: (a -> a -> a) -> [a] -> a
23    myFoldl1 f (x:xs) = foldl f x xs
24
25    myFoldr1 :: (a -> a -> a) -> [a] -> a
26    myFoldr1 f [x] = x
27    myFoldr1 f (x:xs) = f x (myFoldr1 f xs)
28
29    myTakenWhile :: (a -> Bool) -> [a] -> [a]
30    myTakenWhile pred xs = foldr func [] xs
31    where func x acc = if pred x then x : acc else []
32
33    mySpan :: (a -> Bool) -> [a] -> ([a], [a])
34    mySpan pred xs = foldr func ([], []) xs
35    where
36    func x (truePart, falsePart)
37     pred x = (x: truePart, falsePart)
38     otherwise = ([], x: truePart ++ falsePart)
39
40    myMaybe :: b -> (a -> b) -> Maybe a -> b
41    myMaybe x \_ Nothing = x
42    myMaybe \_ f (Just y) = f y
43
44    data MyList a = MyEmpty MyCons a (MyList a) deriving Show
45
46    myMap :: (a -> b) -> MyList a -> MyList b
47    myMap \_ MyEmpty = MyEmpty
48    myMap f (MyCons x xs) = MyCons (f x) (myMap f xs)
49
50    myUnFoldr :: (b -> Maybe (a, b)) -> b -> [a]
51    myUnFoldr f value = doing (f value)
52    where
53    doing Nothing = []
54    doing (Just (x, newValue)) = x : myUnFoldr f newValue
55
56    newtype Grams = Grams Double deriving (Show)
57    newtype Pieces = Pieces Int deriving (Show)
58    newtype Milliliters = Milliliters Double deriving (Show)
```

```
59
60    data Ingredient = Oil Grams
61     Chocolate Grams
62     Egg Pieces
63     Flour Grams
64     Sugar Grams
65     BakingPowder Grams
66     Milk Milliliters
67     Cream Milliliters
68     Lemon Grams
69     LemonZest Grams
70     Matcha Grams
71     CreamCheese Grams
72     Honey Grams
73     CondensedMilk Milliliters
74     Butter Grams
75     Kefir Milliliters
76     Cocoa Grams
77     FoodColoring String
78     Vinegar Milliliters
79     Salt Grams
80     VanillaSugar Grams
81    deriving (Show)
82
83    data CakeType = ChocolateCake
84     LemonCake
85     MatchaCheesecake
86     HoneyCake
87     RedVelvet
88    deriving (Show, Eq)
89
90    data Dough = PlainDough
91     ChocolateDough
92     LemonDough
93     MatchaDough
94     HoneyDough
95     RedVelvetDough
96    deriving (Show)
97
98    data Cake = Cake CakeType deriving (Show)
99
100   data Oven = Oven Int deriving (Show)
101
102   isOil (Oil \_) = True
103   isOil \_ = False
104   isChocolate (Chocolate \_) = True
105   isChocolate \_ = False
106   isEgg (Egg \_) = True
107   isEgg \_ = False
108
109   makeChocolateCake :: [Ingredient] -> Either String (Cake,
    String)
110   makeChocolateCake ingredients = ...
111
```

```
112     makeLemonCake :: [Ingredient] -> Either String (Cake, String
    )
113     makeLemonCake ingredients = ...
114
115     makeMatchaCheesecake :: [Ingredient] -> Either String (Cake,
    String)
116     makeMatchaCheesecake ingredients = ...
117
118     makeHoneyCake :: [Ingredient] -> Either String (Cake, String
    )
119     makeHoneyCake ingredients = ...
120
121     makeRedVelvet :: [Ingredient] -> Either String (Cake, String
    )
122     makeRedVelvet ingredients = ...
123
124     bakeCake :: Dough -> Oven -> Either String Cake
125     bakeCake ChocolateDough (Oven temp) =
126     if temp >= 170 \&\& temp <= 180
127     then Right (Cake ChocolateCake)
128     else Left "ERROR: wrong temperature for chocolate cake"
129     bakeCake LemonDough (Oven temp) =
130     if temp >= 160 \&\& temp <= 175
131     then Right (Cake LemonCake)
132     else Left "ERROR: wrong temperature for lemon cake"
133
```

Листинг 2: pr01_2.hs

# Practice Work No. 2 (MyEvolModule.hs)

```
1     module MyEvolModule
2     ( MyEvolution(..)
3     , MyEvolution'(..)
4     ) where
5
6     data MyEvolution = LUCA
7      Cyanobacteria
8      Trilobite
9      Ichthyostega
10     Dimetrodon
11     Archaeopteryx
12     Morganucodon
13     Purgatorius
14     Australopithecine
15     Humans
16
17    instance Show MyEvolution where
18    show LUCA = "Last Universal Common Ancestor"
19    show Cyanobacteria = "Synechococcus"
20    show Trilobite = "Paradoxides"
21    show Ichthyostega = "Ichthyostega"
```

```
22    show Dimetrodon = "Dimetrodon"
23    show Archaeopteryx = "Archaeopteryx"
24    show Morganucodon = "Morganucodon"
25    show Purgatorius = "Purgatorius"
26    show Australopithecine = "Australopithecus Afarensis"
27    show Humans = "Homo Sapiens"
28
29    instance Read MyEvolution where
30    readsPrec \_ str = case str of
31    'L':'a':'s':'t':' ':'U':'n':'i':'v':'e':'r':'s':'a':'l':'
      ':'C':'o':'m':'m':'o':'n':' ':'A':'n':'c':'e':'s':'t':'o':'r
      ':rest -> [(LUCA, rest)]
32    'S':'y':'n':'e':'c':'h':'o':'c':'o':'c':'c':'u':'s':rest ->
      [(Cyanobacteria, rest)]
33    'P':'a':'r':'a':'d':'o':'x':'i':'d':'e':'s':rest -> [(
      Trilobite, rest)]
34    'I':'c':'h':'t':'h':'y':'o':'s':'t':'e':'g':'a':rest -> [(
      Ichthyostega, rest)]
35    'D':'i':'m':'e':'t':'r':'o':'d':'o':'n':rest -> [(Dimetrodon
      , rest)]
36    'A':'r':'c':'h':'a':'e':'o':'p':'t':'e':'r':'y':'x':rest ->
      [(Archaeopteryx, rest)]
37    'M':'o':'r':'g':'a':'n':'u':'c':'o':'d':'o':'n':rest -> [(
      Morganucodon, rest)]
38    'P':'u':'r':'g':'a':'t':'o':'r':'i':'u':'s':rest -> [(
      Purgatorius, rest)]
39    'A':'u':'s':'t':'r':'a':'l':'o':'p':'i':'t':'h':'e':'c':'u
      ':'s':' ':'A':'f':'a':'r':'e':'n':'s':'i':'s':rest -> [(
      Australopithecine, rest)]
40    'H':'o':'m':'o':' ':'S':'a':'p':'i':'e':'n':'s':rest -> [(
      Humans, rest)]
41    \_ -> []
42
43    instance Eq MyEvolution where
44    (==) LUCA LUCA = True
45    (==) Cyanobacteria Cyanobacteria = True
46    (==) Trilobite Trilobite = True
47    (==) Ichthyostega Ichthyostega = True
48    (==) Dimetrodon Dimetrodon = True
49    (==) Archaeopteryx Archaeopteryx = True
50    (==) Morganucodon Morganucodon = True
51    (==) Purgatorius Purgatorius = True
52    (==) Australopithecine Australopithecine = True
53    (==) Humans Humans = True
54    (==) \_ \_ = False
55
56    instance Ord MyEvolution where
57    compare LUCA LUCA = EQ
58    compare LUCA \_ = LT
59    compare \_ LUCA = GT
60    compare Cyanobacteria Cyanobacteria = EQ
61    compare Cyanobacteria \_ = LT
62    compare \_ Cyanobacteria = GT
63    compare Trilobite Trilobite = EQ
```

```
64      compare Trilobite \_ = LT
65      compare \_ Trilobite = GT
66      compare Ichthyostega Ichthyostega = EQ
67      compare Ichthyostega \_ = LT
68      compare \_ Ichthyostega = GT
69      compare Dimetrodon Dimetrodon = EQ
70      compare Dimetrodon \_ = LT
71      compare \_ Dimetrodon = GT
72      compare Archaeopteryx Archaeopteryx = EQ
73      compare Archaeopteryx \_ = LT
74      compare \_ Archaeopteryx = GT
75      compare Morganucodon Morganucodon = EQ
76      compare Morganucodon \_ = LT
77      compare \_ Morganucodon = GT
78      compare Purgatorius Purgatorius = EQ
79      compare Purgatorius \_ = LT
80      compare \_ Purgatorius = GT
81      compare Australopithecine Australopithecine = EQ
82      compare Australopithecine \_ = LT
83      compare \_ Australopithecine = GT
84      compare Humans Humans = EQ
85
86      instance Enum MyEvolution where
87      toEnum 0 = LUCA
88      toEnum 1 = Cyanobacteria
89      toEnum 2 = Trilobite
90      toEnum 3 = Ichthyostega
91      toEnum 4 = Dimetrodon
92      toEnum 5 = Archaeopteryx
93      toEnum 6 = Morganucodon
94      toEnum 7 = Purgatorius
95      toEnum 8 = Australopithecine
96      toEnum 9 = Humans
97      toEnum n = error $ "toEnum: no such constructor " ++ show n
98      fromEnum LUCA = 0
99      fromEnum Cyanobacteria = 1
100     fromEnum Trilobite = 2
101     fromEnum Ichthyostega = 3
102     fromEnum Dimetrodon = 4
103     fromEnum Archaeopteryx = 5
104     fromEnum Morganucodon = 6
105     fromEnum Purgatorius = 7
106     fromEnum Australopithecine = 8
107     fromEnum Humans = 9
108
109     instance Bounded MyEvolution where
110     minBound = LUCA
111     maxBound = Humans
112
113     data MyEvolution' = LUCA'
114      Cyanobacteria'
115      Trilobite'
116      Ichthyostega'
117      Dimetrodon'
```

```
118      Archaeopteryx'
119      Morganucodon'
120      Purgatorius'
121      Australopithecine'
122      Humans'
123    deriving (Show, Read, Eq, Ord, Enum, Bounded)
124
```

Листинг 3: MyEvolModule.hs

# Practice Work No. 2 (MyMaybe.hs)

```
1    module MyTypes.MyMaybe (MyMaybe(..)) where
2
3    data MyMaybe a = MyNothing MyJust a deriving (Show, Eq, Ord)
4
5    instance Foldable MyMaybe where
6    foldr \_ z MyNothing = z
7    foldr f z (MyJust x) = f x z
8
9    instance Semigroup a => Semigroup (MyMaybe a) where
10   MyNothing <> b = b
11   a <> MyNothing = a
12   (MyJust x) <> (MyJust y) = MyJust (x <> y)
13
14   instance Semigroup a => Monoid (MyMaybe a) where
15   mempty = MyNothing
16   mappend = (<>)
17
18   instance Functor MyMaybe where
19   fmap \_ MyNothing = MyNothing
20   fmap f (MyJust x) = MyJust (f x)
21
22   instance Applicative MyMaybe where
23   pure = MyJust
24   MyNothing <*> \_ = MyNothing
25   \_ <*> MyNothing = MyNothing
26   (MyJust f) <*> (MyJust x) = MyJust (f x)
27
```

Листинг 4: MyMaybe.hs

# Practice Work No. 2 (MyEither.hs)

```
1    module MyTypes.MyEither (MyEither(..)) where
2
3    data MyEither a b = MyLeft a MyRight b deriving (Show, Eq,
   Ord)
4
5    instance Foldable (MyEither a) where
```

```
6     foldr \_ z (MyLeft \_) = z
7     foldr f z (MyRight y) = f y z
8
9     instance Semigroup b => Semigroup (MyEither a b) where
10    (MyLeft x) <> \_ = MyLeft x
11    \_ <> (MyLeft y) = MyLeft y
12    (MyRight x) <> (MyRight y) = MyRight (x <> y)
13
14    instance Functor (MyEither a) where
15    fmap \_ (MyLeft x) = MyLeft x
16    fmap f (MyRight y) = MyRight (f y)
17
18    instance Applicative (MyEither a) where
19    pure = MyRight
20    (MyLeft x) <*> \_ = MyLeft x
21    \_ <*> (MyLeft y) = MyLeft y
22    (MyRight f) <*> (MyRight x) = MyRight (f x)
23
```

Листинг 5: MyEither.hs

# Practice Work No. 2 (MyTree.hs)

```
1     module MyTypes.MyTree (MyTree(..)) where
2
3     data MyTree a = Leaf a Node a (MyTree a) (MyTree a) deriving
      (Show, Eq)
4
5     instance Foldable MyTree where
6     foldMap f (Leaf x) = f x
7     foldMap f (Node x left right) = foldMap f left <> f x <>
    foldMap f right
8
9     instance Functor MyTree where
10    fmap f (Leaf x) = Leaf (f x)
11    fmap f (Node x left right) = Node (f x) (fmap f left) (fmap
    f right)
12
13    instance Applicative MyTree where
14    pure x = Leaf x
15    (Leaf f) <*> tree = fmap f tree
16    (Node f leftf rightf) <*> tree =
17    Node (f (getValue tree))
18    (leftf <*> tree)
19    (rightf <*> tree)
20    where
21    getValue (Leaf x) = x
22    getValue (Node x \_ \_) = x
23
```

Листинг 6: MyTree.hs

# Practice Work No. 2 (MyParser.hs)

```
1     module MyParsers.MyParser (
2     Parser, runParser, satisfy, char, digit, digits, plusOrMult
3     ) where
4
5     import MyTypes.MyMaybe
6     import Data.Char (isDigit)
7     import Control.Applicative (Alternative(empty, (<>)))
8
9     newtype Parser tok a = Parser {runParser :: [tok] -> MyMaybe
      ([tok], a)}
10
11    instance Functor (Parser tok) where
12    fmap g (Parser u) = Parser f where
13    f xs = case u xs of
14    MyNothing -> MyNothing
15    MyJust (toks', x) -> MyJust (toks', g x)
16
17    instance Applicative (Parser tok) where
18    pure x = Parser xs -> MyJust (xs, x)
19    Parser u <*> Parser v = Parser f where
20    f xs = case u xs of
21    MyNothing -> MyNothing
22    MyJust (toks, x) -> case v toks of
23    MyNothing -> MyNothing
24    MyJust (toks', x') -> MyJust (toks', x x')
25
26    instance Alternative (Parser tok) where
27    empty = Parser $ \_ -> MyNothing
28    Parser u Parser v = Parser f where
29    f xs = case u xs of
30    MyNothing -> v xs
31    MyJust x -> MyJust x
32
33    satisfy :: (tok -> Bool) -> Parser tok tok
34    satisfy p = Parser f where
35    f (x:xs)  p x = MyJust (xs, x)
36    f \_ = MyNothing
37
38    char :: Char -> Parser Char Char
39    char c = satisfy (== c)
40
41    digit :: Parser Char Int
42    digit = (c -> digitToInt c) <$> satisfy isDigit
43    where
44    digitToInt c = fromEnum c - fromEnum '0'
45
46    digits :: Parser Char Int
47    digits = foldl (acc d -> acc * 10 + d) 0 <$> some digit
48    where
49    some p = (:) <$> p <*> many p
50    many p = some p pure []
51
```

```
52    plusOrMult :: Parser Char Int
53    plusOrMult = mult plus
54    where
55    mult = (*) <$> digits <* char '*' <*> digits
56    plus = (+) <$> digits <* char '+' <*> digits
57
```

Листинг 7: MyParser.hs

# Practice Work No. 2 (ParsecParser.hs)

```
1     module MyParsers.ParsecParser (
2     digitParsec, digitsParsec, multParsec, plusParsec,
3     plusOrMultParsec, runParserParsec
4     ) where
5
6     import Text.Parsec
7     import Text.Parsec.String (Parser)
8     import Data.Char (isDigit, digitToInt)
9     import MyTypes.MyMaybe
10    import qualified Control.Applicative as A
11
12    digitParsec :: Parser Int
13    digitParsec = digitToInt <$> satisfy isDigit
14
15    digitsParsec :: Parser Int
16    digitsParsec = fmap (foldl (\x y -> x * 10 + y) 0) (many1
    digitParsec)
17
18    multParsec :: Parser Int
19    multParsec = (*) <$> digitsParsec <* char '*' <*>
    digitsParsec
20
21    plusParsec :: Parser Int
22    plusParsec = (+) <$> digitsParsec <* char '+' <*>
    digitsParsec
23
24    plusOrMultParsec :: Parser Int
25    plusOrMultParsec = multParsec A.<> plusParsec
26
27    runParserParsec :: Parser Int -> String -> MyMaybe Int
28    runParserParsec p input =
29    case parse p "" input of
30    Left \_ -> MyNothing
31    Right result -> MyJust result
32
```

Листинг 8: ParsecParser.hs

# Practice Work No. 2 (AttoparsecParser.hs)

```haskell
module MyParsers.AttoparsecParser (
digitAttoparsec, digitsAttoparsec, multAttoparsec,
plusAttoparsec, plusOrMultAttoparsec, runParserAttoparsec
) where

import Data.Attoparsec.Text
import qualified Data.Text as T
import Data.Char (isDigit, digitToInt)
import MyTypes.MyMaybe
import qualified Control.Applicative as A

digitAttoparsec :: Parser Int
digitAttoparsec = digitToInt <$> satisfy isDigit

digitsAttoparsec :: Parser Int
digitsAttoparsec = foldl (\acc d -> acc * 10 + d) 0 <$> many1 digitAttoparsec

multAttoparsec :: Parser Int
multAttoparsec = (*) <$> digitsAttoparsec <* char '*' <*> digitsAttoparsec

plusAttoparsec :: Parser Int
plusAttoparsec = (+) <$> digitsAttoparsec <* char '+' <*> digitsAttoparsec

plusOrMultAttoparsec :: Parser Int
plusOrMultAttoparsec = multAttoparsec A.<> plusAttoparsec

runParserAttoparsec :: Parser Int -> String -> MyMaybe Int
runParserAttoparsec p input =
case parseOnly p (T.pack input) of
Left \_ -> MyNothing
Right result -> MyJust result

```

Листинг 9: AttoparsecParser.hs

# Practice Work No. 3 (Glitch.hs)

```haskell
{-# LANGUAGE OverloadedStrings #-}

module Glitch
( replaceByte
, sortSection
, reverseSection
, randomReplaceByte
, randomSortSection
, randomReverseBytes
, applyRandomReplacements
, applyRandomSorts
```

```
12    , applyRandomReverses
13    , applyFullGlitch
14    , applyFullGlitchWithReverse
15    ) where
16
17    import qualified Data.ByteString.Char8 as BC
18    import System.Random (randomRIO)
19    import Control.Monad (foldM)
20
21    replaceByte :: Int -> Int -> BC.ByteString -> BC.ByteString
22    replaceByte pos newVal bytes = mconcat [before, newChar,
      after]
23    where
24    (before, rest) = BC.splitAt pos bytes
25    after = BC.drop 1 rest
26    newChar = BC.pack [toEnum (newVal \`mod\` 255)]
27
28    sortSection :: Int -> Int -> BC.ByteString -> BC.ByteString
29    sortSection start size bytes = mconcat [before, changed,
      after]
30    where
31    (before, rest) = BC.splitAt start bytes
32    (target, after) = BC.splitAt size rest
33    changed = BC.sort target
34
35    reverseSection :: Int -> Int -> BC.ByteString -> BC.
      ByteString
36    reverseSection start size bytes = mconcat [before, changed,
      after]
37    where
38    (before, rest) = BC.splitAt start bytes
39    (target, after) = BC.splitAt size rest
40    changed = BC.reverse target
41
42    randomReplaceByte :: BC.ByteString -> IO BC.ByteString
43    randomReplaceByte bytes = do
44    let len = BC.length bytes
45    if len <= 1
46    then return bytes
47    else do
48    pos <- randomRIO (1, len - 1)
49    newVal <- randomRIO (0, 255)
50    return (replaceByte pos newVal bytes)
51
52    randomSortSection :: BC.ByteString -> IO BC.ByteString
53    randomSortSection bytes = do
54    let len = BC.length bytes
55    sectionSize = 25
56    if len <= sectionSize
57    then return bytes
58    else do
59    start <- randomRIO (0, len - sectionSize)
60    return (sortSection start sectionSize bytes)
61
```

```
62    randomReverseBytes :: BC.ByteString -> IO BC.ByteString
63    randomReverseBytes bytes = do
64    let len = BC.length bytes
65    sectionSize = 25
66    if len <= sectionSize
67    then return bytes
68    else do
69    start <- randomRIO (0, len - sectionSize)
70    return (reverseSection start sectionSize bytes)
71
72    applyRandomReplacements :: BC.ByteString -> Int -> IO BC.
    ByteString
73    applyRandomReplacements bytes 0 = return bytes
74    applyRandomReplacements bytes n = do
75    newBytes <- randomReplaceByte bytes
76    applyRandomReplacements newBytes (n - 1)
77
78    applyRandomSorts :: BC.ByteString -> Int -> IO BC.ByteString
79    applyRandomSorts bytes 0 = return bytes
80    applyRandomSorts bytes n = do
81    newBytes <- randomSortSection bytes
82    applyRandomSorts newBytes (n - 1)
83
84    applyRandomReverses :: BC.ByteString -> Int -> IO BC.
    ByteString
85    applyRandomReverses bytes 0 = return bytes
86    applyRandomReverses bytes n = do
87    newBytes <- randomReverseBytes bytes
88    applyRandomReverses newBytes (n - 1)
89
90    applyFullGlitch :: BC.ByteString -> IO BC.ByteString
91    applyFullGlitch bytes = foldM (\b f -> f b) bytes
92    [ randomReplaceByte
93    , randomSortSection
94    , randomReplaceByte
95    , randomSortSection
96    , randomReplaceByte
97    ]
98
99    applyFullGlitchWithReverse :: BC.ByteString -> IO BC.
    ByteString
100   applyFullGlitchWithReverse bytes = foldM (\b f -> f b) bytes
101   [ randomReplaceByte
102   , randomReverseBytes
103   , randomReplaceByte
104   , randomReverseBytes
105   , randomReplaceByte
106   ]
107
```

Листинг 10: Glitch.hs

# Practice Work No. 3 (Labirint.hs)

```haskell
{-# LANGUAGE FlexibleContexts #-}
{-# LANGUAGE BangPatterns #-}

module Labirint (
Room(..), Maze, MazeState(..), MazeLog(..),
MazeRWS, findPath, solveMaze, labirint1
) where

import Control.Monad.RWS (RWS, runRWS, ask, tell, get, put)

data Room = Room {
  roomName :: String
  , roomDescription :: String
  , connections :: [(String, String)]
} deriving (Show, Eq)

type Maze = [(String, Room)]

data MazeState = MazeState {
  currentRoom :: String
  , visitedRooms :: [String]
  , path :: [String]
  , steps :: Int
} deriving (Show)

newtype MazeLog = MazeLog { getLog :: [String] } deriving (Show)

instance Semigroup MazeLog where
(MazeLog l1) <> (MazeLog l2) = MazeLog (l1 ++ l2)

instance Monoid MazeLog where
mempty = MazeLog []
mappend = (<>)

type MazeRWS a = RWS Maze MazeLog MazeState a

getCurrentRoom :: MazeRWS String
getCurrentRoom = fmap currentRoom get

getVisitedRooms :: MazeRWS [String]
getVisitedRooms = fmap visitedRooms get

isVisited :: String -> MazeRWS Bool
isVisited room = getVisitedRooms >>= \visited -> return (room \`elem\` visited)

markVisited :: String -> MazeRWS ()
markVisited room = get >>= \s ->
put (s { visitedRooms = room : visitedRooms s
  , path = path s ++ [room]
  , currentRoom = room
```

```
51      , steps = steps s + 1 })
52
53    logMovement :: String -> String -> String -> MazeRWS ()
54    logMovement from to direction = get >>= \s ->
55    let msg = "Step " ++ show (steps s) ++ ": " ++ direction
56    ++ " from '" ++ from ++ "' to '" ++ to ++ "'"
57    in tell (MazeLog [msg])
58
59    getRoom :: String -> MazeRWS Room
60    getRoom name = ask >>= \maze ->
61    case lookup name maze of
62    Just room -> return room
63    Nothing -> error $ "Room not found: " ++ name
64
65    getNeighbors :: String -> MazeRWS [(String, String)]
66    getNeighbors roomName = getRoom roomName >>= return .
    connections
67
68    moveToRoom :: String -> String -> MazeRWS Bool
69    moveToRoom from to =
70    getNeighbors from >>= \neighbors ->
71    let canMove = any (\(_, room) -> room == to) neighbors
72    in if canMove
73    then let dir = lookup to (map swap neighbors)
74    in maybe (return ()) (logMovement from to) dir
75    >> markVisited to
76    >> return True
77    else return False
78    where swap (a,b) = (b,a)
79
80    findPath :: String -> String -> MazeRWS Bool
81    findPath start finish = markVisited start >>
    findPathFromCurrent finish
82
83    findPathFromCurrent :: String -> MazeRWS Bool
84    findPathFromCurrent finish = getCurrentRoom >>= \cur ->
85    if cur == finish
86    then tell (MazeLog ["FINISH reached: " ++ finish]) >> return
     True
87    else getVisitedRooms >>= \visited ->
88    getNeighbors cur >>= \neighbors ->
89    let unvisited = filter (\(_, r) -> r \`notElem\` visited)
    neighbors
90    in tryPaths cur unvisited finish
91
92    tryPaths :: String -> [(String, String)] -> String ->
    MazeRWS Bool
93    tryPaths from [] finish =
94    tell (MazeLog ["Dead end from '" ++ from ++ "', backtracking
    ..."]) >> return False
95    tryPaths from ((dir, nextRoom):rest) finish =
96    moveToRoom from nextRoom >>= \success ->
97    if success
98    then findPathFromCurrent finish >>= \found ->
```

```
99    if found then return True
100   else get >>= \s ->
101   tell (MazeLog ["Backtrack from '" ++ nextRoom
102   ++ "', trying next path from '" ++ from ++ "'"]) >>
103   tryPaths from rest finish
104   else tryPaths from rest finish
105
106   solveMaze :: Maze -> String -> String -> IO ()
107   solveMaze maze start finish = do
108   let initialState = MazeState start [] [] 0
109   let (found, finalState, log) = runRWS (findPath start finish
      ) maze initialState
110   putStrLn ""
111   putStrLn "========================================"
112   putStrLn "LABYRINTH PATH FINDER"
113   putStrLn "========================================"
114   putStrLn ("Start: " ++ start)
115   putStrLn ("Finish: " ++ finish)
116   putStrLn ("Path found: " ++ if found then "YES" else "NO")
117   putStrLn ("Steps: " ++ show (steps finalState))
118   putStrLn ""
119   putStrLn "Path taken:"
120   putStrLn ("  " ++ show (path finalState))
121   putStrLn ""
122   putStrLn "Movement log:"
123   mapM_ (\x -> putStrLn ("  " ++ x)) (getLog log)
124   putStrLn "========================================"
125
126   labirint1 :: Maze
127   labirint1 =
128   [ ("start", Room "start" "Mondstadt City Gate" [("east", "
      dragonspine")])
129   , ("dragonspine", Room "dragonspine" "Frozen Mountain Peak"
130   [("west", "start"), ("south", "liyue_harbor")])
131   , ("liyue_harbor", Room "liyue_harbor" "Busy Harbor Port"
132   [("south", "qingce_village"), ("east", "jueyun_karst"), ("
      north", "dragonspine")])
133   , ("qingce_village", Room "qingce_village" "Bamboo Forest
      Village"
134   [("north", "liyue_harbor"), ("south", "chasm")])
135   , ("jueyun_karst", Room "jueyun_karst" "Mysterious Mountain
      Range"
136   [("west", "liyue_harbor"), ("south", "inazuma_city")])
137   , ("chasm", Room "chasm" "Deep Underground Mines"
138   [("north", "qingce_village"), ("east", "inazuma_city")])
139   , ("inazuma_city", Room "inazuma_city" "Electro Archon's
      Capital"
140   [("south", "narukami_shrine"), ("north", "jueyun_karst"), ("
      west", "chasm")])
141   , ("narukami_shrine", Room "narukami_shrine" "Sacred Shrine
      on the Mountain"
142   [("west", "abandoned_ruins"), ("south", "finish"), ("north",
       "inazuma_city")])
```

```
143     , ("abandoned_ruins", Room "abandoned_ruins" "Abandoned
      Ruins of Sal Vindagnyr"
144     [("east", "narukami_shrine")])
145     , ("finish", Room "finish" "Exit from Labyrinth" [("north",
      "narukami_shrine")])
146     ]
147
```

Листинг 11: Labirint.hs

# Practice Work No. 4 (MyStateT.hs)

```
1     {-# LANGUAGE InstanceSigs #-}
2     {-# LANGUAGE FlexibleInstances #-}
3     {-# LANGUAGE MultiParamTypeClasses #-}
4     {-# LANGUAGE FunctionalDependencies #-}
5     {-# LANGUAGE UndecidableInstances #-}
6
7     module MyStateT where
8
9     import Control.Monad (ap)
10    import Control.Monad.Trans.Class (MonadTrans(..))
11    import Control.Monad.IO.Class (MonadIO(..))
12
13    newtype MyStateT s m a = MyStateT { runMyStateT :: s -> m (a
      , s) }
14
15    state :: Monad m => (s -> (a, s)) -> MyStateT s m a
16    state f = MyStateT $ return . f
17
18    get :: Monad m => MyStateT s m s
19    get = state $ \s -> (s, s)
20
21    put :: Monad m => s -> MyStateT s m ()
22    put s = state $ \_ -> ((), s)
23
24    modify :: Monad m => (s -> s) -> MyStateT s m ()
25    modify f = state $ \s -> ((), f s)
26
27    runState :: Monad m => MyStateT s m a -> s -> m (a, s)
28    runState = runMyStateT
29
30    evalState :: Monad m => MyStateT s m a -> s -> m a
31    evalState m s = do
32    (a, \_) <- runMyStateT m s
33    return a
34
35    execState :: Monad m => MyStateT s m a -> s -> m s
36    execState m s = do
37    (\_, s') <- runMyStateT m s
38    return s'
39
```

```
40      instance Monad m => Functor (MyStateT s m) where
41      fmap f m = MyStateT $ \s -> do
42      (a, s') <- runMyStateT m s
43      return (f a, s')
44
45      instance Monad m => Applicative (MyStateT s m) where
46      pure a = MyStateT $ \s -> return (a, s)
47      (<*>) = ap
48
49      instance Monad m => Monad (MyStateT s m) where
50      m >>= k = MyStateT $ \s -> do
51      (a, s') <- runMyStateT m s
52      runMyStateT (k a) s'
53
54      instance MonadTrans (MyStateT s) where
55      lift ma = MyStateT $ \s -> do
56      a <- ma
57      return (a, s)
58
59      instance MonadIO m => MonadIO (MyStateT s m) where
60      liftIO = lift . liftIO
61
```

Листинг 12: MyStateT.hs

# Practice Work No. 4 (GameMonad.hs)

```
1       {-# LANGUAGE FlexibleContexts #-}
2
3       module GameMonad (
4       GameState(..), GameLog, GameMonad,
5       runGame, getCurrentRoomInfo, logMessage,
6       moveDirection, getAvailableDirections,
7       isFinish, processCommand, initGameState
8       ) where
9
10      import MyStateT
11      import MazeTypes
12      import Control.Monad.Reader (ReaderT, ask, runReaderT)
13      import Control.Monad.Writer (WriterT, tell, runWriterT)
14      import Control.Monad.Trans.Class (lift)
15      import qualified Data.Map as Map
16
17      data GameState = GameState {
18        currentRoomName :: String
19        , visitedRooms :: [String]
20        , movesCount :: Int
21      } deriving (Show)
22
23      type GameLog = [String]
24
```

```
25    type GameMonad a = MyStateT GameState (WriterT GameLog (
    ReaderT Maze IO)) a
26
27    runGame :: GameMonad a -> Maze -> GameState -> IO (a,
    GameState, GameLog)
28    runGame game maze st = do
29    ((a, state), log) <- runReaderT (runWriterT (runMyStateT
    game st)) maze
30    return (a, state, log)
31
32    getMaze :: GameMonad Maze
33    getMaze = lift $ lift ask
34
35    getCurrentRoomInfo :: GameMonad Room
36    getCurrentRoomInfo = do
37    st <- get
38    maze <- getMaze
39    case Map.lookup (currentRoomName st) maze of
40    Just room -> return room
41    Nothing -> error $ "Room not found: " ++ currentRoomName st
42
43    logMessage :: String -> GameMonad ()
44    logMessage msg = lift $ tell [msg]
45
46    moveDirection :: String -> GameMonad Bool
47    moveDirection dir = do
48    st <- get
49    let current = currentRoomName st
50    currentRoom <- getCurrentRoomInfo
51    let targetRoom = case dir of
52    "north" -> north currentRoom
53    "south" -> south currentRoom
54    "west" -> west currentRoom
55    "east" -> east currentRoom
56    \_ -> Nothing
57    case targetRoom of
58    Just roomName -> do
59    let newState = st {
60      currentRoomName = roomName,
61      visitedRooms = roomName : visitedRooms st,
62      movesCount = movesCount st + 1
63    }
64    put newState
65    logMessage $ dir ++ " from " ++ current ++ " to " ++
    roomName
66    return True
67    Nothing -> do
68    logMessage $ "Cannot go " ++ dir ++ " from " ++ current
69    return False
70
71    getAvailableDirections :: GameMonad [(String, String)]
72    getAvailableDirections = do
73    room <- getCurrentRoomInfo
74    return $ neighbors room
```

```
75
76      isFinish :: String -> GameMonad Bool
77      isFinish finishRoom = do
78      st <- get
79      return $ currentRoomName st == finishRoom
80
81      processCommand :: String -> GameMonad Bool
82      processCommand cmd = case cmd of
83      "north" -> moveDirection "north"
84      "south" -> moveDirection "south"
85      "west" -> moveDirection "west"
86      "east" -> moveDirection "east"
87      "quit" -> return True
88      \_ -> return False
89
90      initGameState :: String -> GameState
91      initGameState startRoom = GameState startRoom [startRoom] 0
92
```

Листинг 13: GameMonad.hs

# Practice Work No. 5 (Lib.hs)

```
1       module Lib
2       ( addMod
3       , reverseWords
4       ) where
5
6       addMod :: Int -> Int -> Int -> Int
7       addMod x y m = (x + y) \`mod\` m
8
9       reverseWords :: String -> String
10      reverseWords = unwords . reverse . words
11
```

Листинг 14: Lib.hs

# Practice Work No. 5 (Spec.hs)

```
1       module Main where
2
3       import Test.QuickCheck
4       import Lib
5
6       genWord :: Gen String
7       genWord = listOf1 (elements ['a'..'z'])
8
9       genSentence :: Gen String
10      genSentence = do
11      wordsList <- listOf1 genWord
```

```
12    return $ unwords wordsList
13
14    prop_addMod_correct :: Int -> Int -> Positive Int -> Bool
15    prop_addMod_correct x y (Positive m) =
16    addMod x y m == (x + y) \`mod\` m
17
18    prop_addMod_zero :: Int -> Positive Int -> Bool
19    prop_addMod_zero x (Positive m) =
20    addMod x 0 m == x \`mod\` m
21
22    prop_addMod_commutative :: Int -> Int -> Positive Int ->
    Bool
23    prop_addMod_commutative x y (Positive m) =
24    addMod x y m == addMod y x m
25
26    prop_rev_empty :: Bool
27    prop_rev_empty = reverseWords "" == ""
28
29    prop_rev_singleWord :: Property
30    prop_rev_singleWord = forAll genWord $ \word ->
31    reverseWords word == word
32
33    prop_rev_multiple :: Property
34    prop_rev_multiple = forAll genSentence $ \sentence ->
35    let wordsList = words sentence
36    in reverseWords sentence == unwords (reverse wordsList)
37
38    prop_rev_involution :: Property
39    prop_rev_involution = forAll genSentence $ \sentence ->
40    reverseWords (reverseWords sentence) == sentence
41
42    main :: IO ()
43    main = do
44    putStrLn "Testing addMod"
45    quickCheck prop_addMod_correct
46    quickCheck prop_addMod_zero
47    quickCheck prop_addMod_commutative
48    putStrLn "\nTesting reverseWords"
49    quickCheck prop_rev_empty
50    quickCheck prop_rev_singleWord
51    quickCheck prop_rev_multiple
52    quickCheck prop_rev_involution
53    putStrLn "\nAll tests passed"
54
```

Листинг 15: Spec.hs